

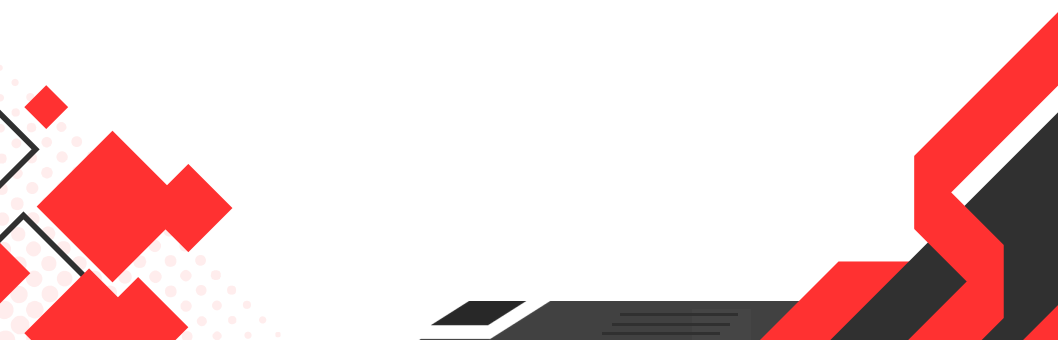


PracticAI

RAG

Commonly Confused Term Pairs →

Terms that people think are the same but are actually different when building RAG systems.



1. Embedding vs Vectorization

What people think:

"Same thing - converting text to numbers"

Reality:

- **Embedding** = Creating semantic representations using neural networks (captures meaning)
- **Vectorization** = Any conversion of text to numerical format (includes simple methods like TF-IDF, one-hot encoding)

Example:

Vectorization: "cat" $\rightarrow$ [0, 0, 1, 0, 0] (one-hot, no meaning)

Embedding: "cat" $\rightarrow$ [0.2, -0.8, 0.3, ...] (learned representation, captures "animal", "pet", etc.)

Why it matters:

All embeddings are vectorization, but not all vectorization is embedding. RAG needs embeddings specifically.

2. Similarity vs Relevance

What people think: "If it's similar, it's relevant"

Reality:

- **Similarity** = Mathematical closeness in vector space (cosine score)
- **Relevance** = Actually useful for answering the query

Example:

Query: "How to fix login error code 401?"

- High similarity (0.92): "Understanding HTTP status codes" - similar topic, not relevant
- Lower similarity (0.78): "Login troubleshooting: 401 unauthorized fix" - highly relevant!

Why it matters: You can have high similarity scores but low relevance. Don't rely solely on similarity thresholds.

3. Chunk vs Document

What people think: "Just different sizes of text"

Reality:

- **Chunk** = A piece of a document created for retrieval purposes (can overlap, have metadata linking to parent)
- **Document** = The original complete source (PDF, article, page)

Example:

- Document: 50-page employee handbook
- Chunks: 200 overlapping 500-token pieces with metadata pointing back to the handbook

Why it matters: You retrieve chunks but might need to show the full document. They have different lifecycles and purposes.

4. Index vs Database

4

What people think: "Vector database and index are the same"

Reality:

- **Index** = The search structure (HNSW graph, IVF clusters) for fast retrieval
- **Database** = Storage system that may contain multiple indices, metadata, documents, etc.

Example:

- Database: Pinecone/Weaviate storing IM vectors + metadata + manages connections
- Index: HNSW algorithm structure within that database for searching

Why it matters: You can rebuild indices without losing data. Database failures lose everything. Different optimization strategies.

5. Latency vs Response Time

What people think: "Same - how long the user waits"

Reality:

- **Latency** = Time waiting before processing starts (network delay, queuing)
- **Response Time** = Total time from request to complete answer (includes latency + processing)

Example:

- Latency: 50ms (request reaches server, waits in queue)
- Embedding query: 30ms
- Vector search: 100ms
- LLM generation: 2000ms
- Response time: 2180ms total

Why it matters: You can optimize latency (better networking, reduce queue) separately from processing time (faster models, better hardware).

6. Retrieval Precision vs Recall

6

What people think: "Both mean getting the right documents"

Reality:

- **Precision** = Of what you retrieved, how much was relevant (quality of results)
- **Recall** = Of all relevant docs, how many did you retrieve (completeness)

Example:

10 relevant docs exist, you retrieve 5 docs: 4 relevant, 1 irrelevant

- Precision: 80% (4 out of 5 retrieved were good)
- Recall: 40% (only got 4 out of 10 relevant docs)

Why it matters: High precision = less noise for LLM. High recall = not missing information. Need different fixes for each.

7. Context Window vs Length

7

What people think: "Same thing - how much text the model can handle"

Reality:

- **Context Window** = Maximum capacity of the model (e.g., 128k tokens - fixed by model)
- **Context Length** = Actual amount you're using in a specific request (e.g., 5k tokens - varies per query)

Example:

- Context window: Claude 3.5 Sonnet has 200k token window
- Context length in your request: 3k tokens (system prompt) + 2k tokens (retrieved chunks) + 500 tokens (chat history) = 5.5k tokens used

Why it matters: Window is your limit. Length is your usage. Monitor length to stay under window and control costs.

8. Semantic vs Vector Search

8

What people think: "Same - searching with embeddings"

Reality:

- **Semantic Search** = Searching by meaning (can use vectors, but also other techniques)
- **Vector Search** = Specific technique using embedding vectors and similarity metrics

Example:

- Semantic search: Could use keyword expansion, synonyms, query rewriting, AND vectors
- Vector search: Specifically using cosine similarity between embedding vectors

Why it matters: Vector search is one way to do semantic search, but semantic search is broader. Don't limit yourself to only vectors.

9. Re-ranking vs Re-scoring

9

What people think: "Just sorting results again"

Reality:

- **Re-ranking** = Using a different, usually more sophisticated model to reorder results
- **Re-scoring** = Recalculating scores using the same method or simple rule adjustments

Example:

- Re-ranking: Get 100 results from embeddings → use cross-encoder model to re-evaluate top 20 → reorder
- Re-scoring: Adjust similarity scores based on metadata (boost recent docs by 0.1) → re-sort

Why it matters: Re-ranking is slower but more accurate (different model). Re-scoring is fast but limited (same information).

10. Embedding Model vs LLM

10

What people think: "Both are AI models, can use the same one"

Reality:

- **Embedding Model** = Converts text to vectors (e.g., text-embedding-ada-002, 1536 dimensions)
- **LLM** = Generates text from text (e.g., GPT-4, Claude)

Example:

- Embedding model: "Python tutorial" → [0.23, -0.45, 0.67, ...] (vector)
- LLM: "Python tutorial" → "Python is a programming language..." (text)

Why it matters: Different models, different APIs, different costs. Can't use GPT-4 for embeddings or embedding model for generation.

11. Query vs Document Embedding

11

What people think: "Same process, just different text"

Reality:

- **Query Embedding** = Embedding of user's question (done at query time, needs to be fast)
- **Document Embedding** = Embedding of knowledge base chunks (done offline, can be slower/better)

Example:

- Document embedding: Use best quality model, batch processing, done once, store results
- Query embedding: Use fast model, single request, done every query, not stored

Why it matters: Different optimization strategies. You might use different models or configurations for each.

12. Top-K vs Threshold Retrieval

12

What people think: "Both ways to limit results"

Reality:

- **Top-K** = Always return exactly K results (e.g., top 5), regardless of quality
- **Threshold** = Return all results above quality score (could be 0 results or 100)

Example:

Query has poor matches:

- Top-K ($K=5$): Returns 5 docs with scores: 0.45, 0.42, 0.38, 0.35, 0.31 (all poor but returns anyway)
- Threshold (>0.7): Returns 0 docs (none good enough)

Why it matters: Top-K always returns something (could be garbage). Threshold can return nothing (safer but might frustrate users).

13. Cold Start vs Empty Index

What people think: "System not ready"

Reality:

- **Cold Start** = Index exists but not loaded in memory yet (first query is slow)
- **Empty Index** = No documents indexed yet (can't answer any queries)

Example:

- Cold start: 1M docs indexed, server restart, first query takes 30s to load index
- Empty index: Brand new system, 0 docs, can't answer anything until you index data

Why it matters: Cold start is a performance issue. Empty index is a data issue. Different solutions needed.

14. Metadata vs Semantic Filter

14

What people think: "Both ways to filter search results"

Reality:

- **Metadata Filtering** = Filter by structured fields (category, date, author - exact matches)
- **Semantic Filtering** = Filter by meaning/content (similarity to concepts - fuzzy)

Example:

- Metadata: "Show only docs from 'engineering' category AND after 2024"
- Semantic: "Show only docs semantically similar to 'python programming'"

Why it matters: Metadata is fast, precise, boolean. Semantic is slower, fuzzy, scored. Combine them for best results.

15. Hallucination vs Bad Retrieval

15

What people think: "Wrong answer means the LLM hallucinated"

Reality:

- **Hallucination** = LLM makes up information not in retrieved context
- **Incorrect Retrieval** = Retrieved wrong documents, LLM faithfully uses them

Example:

Query: "What's our refund policy?"

- **Incorrect retrieval**: Retrieved competitor's policy → LLM says "30 days" (wrong source, but faithful)
- **Hallucination**: Retrieved correct policy "14 days" → LLM says "30 days" (right source, made it up)

Why it matters: Different root causes need different fixes. Bad retrieval → fix retrieval. Hallucination → fix prompt/model.

16. Chunking Strategy vs Size

16

What people think: "How to split documents"

Reality:

- **Chunking Strategy** = The method/logic for splitting (semantic, paragraph, sentence, sliding window)
- **Chunking Size** = The target length of chunks (512 tokens, 1000 chars, etc.)

Example:

- Strategy: "Split by paragraphs, keep sentences together"
- Size: "Target 500 tokens per chunk"
- Result: Some chunks 300 tokens, some 700 tokens (strategy respected, size is approximate)

Why it matters: Strategy affects quality. Size affects performance. You need to configure both independently.

17. Dense vs Sparse Vectors

What people think: "Different ways to represent text as numbers"

Reality:

- **Dense Vectors** = Most/all values are non-zero (neural embeddings: [0.23, -0.45, 0.67, 0.12, ...])
- **Sparse Vectors** = Most values are zero (keyword-based: [0, 0, 0.7, 0, 0, 0, 0.3, 0, ...])

Example:

"machine learning":

- Dense (384d): 384 non-zero values capturing semantic meaning
- Sparse (50,000d): 2 non-zero values (positions for "machine" and "learning" in vocabulary)

Why it matters: Dense is better for semantics, sparse is better for exact matches. Hybrid search uses both. Different indexing requirements.

18. Reranking vs Filtering

18

What people think: "Both narrow down results"

Reality:

- **Reranking** = Changing the order of results using a different scoring method
- **Filtering** = Removing results that don't meet criteria (reduces count)

Example:

10 retrieved docs:

- Filtering: Remove docs older than 2024 → now have 6 docs
- Reranking: Keep all 10 but reorder using cross-encoder → still have 10 docs, different order

Why it matters: Filtering reduces results (can combine with metadata). Reranking preserves count but improves order. Often use both together.

19. Indexing vs Inference Time

19

What people think: "Time it takes for RAG to work"

Reality:

- **Indexing Time** = Offline process: chunking, embedding, storing documents (happens once or periodically)
- **Inference Time** = Query time: embedding query, searching, generating answer (happens per request)

Example:

- Indexing: Processing 10k documents takes 2 hours (batch job, offline)
- Inference: Answering one question takes 3 seconds (real-time, user waits)

Why it matters: Can tolerate slow indexing (offline). Must optimize inference (users waiting). Different performance requirements.

20. Context Stuffing vs Optimization

20

What people think: "Adding retrieved docs to the prompt"

Reality:

- **Context Stuffing** = Dumping all retrieved chunks into prompt (lazy approach)
- **Context Optimization** = Selecting, ordering, formatting chunks strategically

Example:

Retrieved 10 chunks:

- *Context stuffing: Paste all 10 in random order → 8k tokens, redundant info*
- *Context optimization: Pick top 3 most relevant, remove overlap, order by importance → 2k tokens, focused info*

Why it matters: *Stuffing wastes tokens and confuses LLM. Optimization improves quality and reduces cost.*

21. Hybrid vs Multi-Vector Search

21

What people think: "Using multiple search methods"

Reality:

- **Hybrid Search** = Combining different search types (dense + sparse/keyword, like vector + BM25)
- **Multi-Vector Search** = Multiple embeddings per document (e.g., ColBERT - one vector per token)

Example:

- Hybrid: Run vector search AND keyword search → merge results
- Multi-vector: Each document has 50 token embeddings → compare each to query

Why it matters: Hybrid combines different algorithms. Multi-vector is one algorithm with more vectors. Different complexity and use cases.

22. RAG vs Fine-tuning

22

What people think: "Ways to add knowledge to LLMs"

Reality:

- **Retrieval Augmentation (RAG)** = Dynamically fetch relevant info at query time, add to prompt
- **Fine-tuning** = Permanently update model weights with training on custom data

Example:

- RAG: User asks question → find relevant docs → add to prompt → generate answer (knowledge stays external)
- Fine-tuning: Train model on company docs → model weights updated → knowledge baked into model

Why it matters: RAG is flexible, updatable, transparent. Fine-tuning is permanent, faster at inference but harder to update. Different use cases.

23. Vector Database vs Index

23

What people think: "Same - where vectors are stored"

Reality:

- **Vector Database** = Full system with storage, multiple indices, metadata, CRUD operations, persistence, API
- **Vector Index** = The search algorithm/structure (like HNSW) for finding similar vectors

Example:

- Vector database: Pinecone (manages data, scaling, APIs, backups, multiple namespaces)
- Vector index: HNSW algorithm (just the search structure, one component of the database)

Why it matters: Index is a component. Database is the whole system. You might have multiple indices in one database.

24. Prompt vs Architecture

24

What people think: "Both ways to improve LLM answers"

Reality:

- **Prompt Engineering** = Crafting the text instructions/format given to LLM
- **RAG Architecture** = The entire system design: chunking, retrieval, reranking, context management

Example:

- Prompt engineering: "Use bullet points, cite sources, think step by step..."
- RAG architecture: Deciding chunk size, overlap, number of retrievals, reranking strategy, metadata schema

Why it matters: Good prompts can't fix bad retrieval. Good retrieval can be ruined by bad prompts. Need both optimized separately.

25. Faithfulness vs Accuracy

25

What people think: "Both mean the answer is correct"

Reality:

- **Faithfulness** = Answer is grounded in retrieved documents (not hallucinated)
- **Accuracy** = Answer is factually correct (matches ground truth)

Example:

Retrieved doc says: "Policy changed in 2023 to 30 days" (but actually it's 14 days)

Answer: "The policy is 30 days"

- Faithfulness: HIGH (answer matches retrieved doc)
- Accuracy: LOW (answer is factually wrong)

Why it matters: Faithful but inaccurate = retrieval problem.
Accurate but unfaithful = unusual. Unfaithful = hallucination problem.

Why These Distinctions Matter

Understanding these differences helps you:

- ✓ **Debug effectively:** Identify whether issues are retrieval, generation, or configuration problems
- ✓ **Optimize correctly:** Apply the right solution to the actual problem
- ✓ **Communicate clearly:** Discuss technical decisions with precision
- ✓ **Avoid costly mistakes:** Don't confuse metrics that require different fixes
- ✓ **Build better systems:** Make informed architectural choices

Remember: In RAG systems, precision in terminology leads to precision in implementation.

Key Insight: Each term pair represents a different layer of the RAG stack. Confusing them means you're optimizing the wrong layer! 🎯